Strutture Iterative (1)

Table of contents

Struttura iterativa `WHILE` 2

Struttura iterativa `WHILE` . . . 2
`WHILE` - sintassi semplificata . . . 5
Schemi logici dei cicli `while` . . . 6
While - Numero di iterazioni noto in anticipo . . . 6
While - Numero di iterazioni non noto in anticipo . . . 7
Obiettivo del ciclo . . . 10
Esempi . . . 11
Calcolo della media dei voti. Il numero di esami è noto. . . . 11
Calcolo della media pesata dei voti. Il numero di esami è noto. . . . 11
Calcolo spesa totale. Criterio di stop: inserisci 0 . . . 12
Calcolo spesa totale - solo importi interi. Criterio di stop: inserisci un carattere . . . 12
Inserimento interi. Criterio di stop: inserisci il carattere `x`. . . . 13
Scrivere un programma che legga 10 numeri e conti quanti sono positivi . . . 14
Scrivere un programma che legga dei voti e conti quanti voti sono sufficienti. Criterio di stop: inserisci un carattere . . . 15
Scrivere un programma che legga dei voti e conti quanti voti sono sufficienti. . . . 15
Esercizi su rilevazione di evento . . . 16
Esercizi su accumulazione . . . 16
Esercizi su Ricerca di estremo . . . 17
Altri Esercizi (non in ordine di difficoltà) . . . 18
Gestire il ciclo con break e continue . . . 19
Uscire dal ciclo con `break` . . . 19
Saltare un'iterazione con `continue` . . . 20
`WHILE` - sintassi completa . . . 20
(IN PROGRESS) . . . 20
Esercizi . . . 20

ESERCIZI SUL WHILE 23

ESERCIZIO . . . 23
ESERCIZIO . . . 23
ESERCIZIO . . . 24

ESERCIZIO . . . 25
ESERCIZIO . . . 26
ESERCIZIO . . . 27
ESERCIZIO . . . 28
ESERCIZIO . . . 28
ESERCIZIO . . . 29

Struttura iterativa FOR **29**
Struttura iterativa FOR . . . 29
FOR . . . 30
Gestire il ciclo con break e continue . . . 30

ESERCIZI SUL FOR **31**
ESERCIZIO . . . 31
ESERCIZIO . . . 31
ESERCIZIO . . . 31
ESERCIZIO . . . 32
ESERCIZIO . . . 32
ESERCIZIO . . . 33
ESERCIZIO . . . 33
ESERCIZIO . . . 33
ESERCIZIO . . . 34
ESERCIZIO . . . 34
ESERCIZIO . . . 34
ESERCIZIO . . . 35
ITERARE SUI DIZIONARI - ESERCIZI . . . 36

Struttura iterativa WHILE

Struttura iterativa WHILE

Spesso abbiamo necessità di ripetere un gruppo di istruzioni varie volte

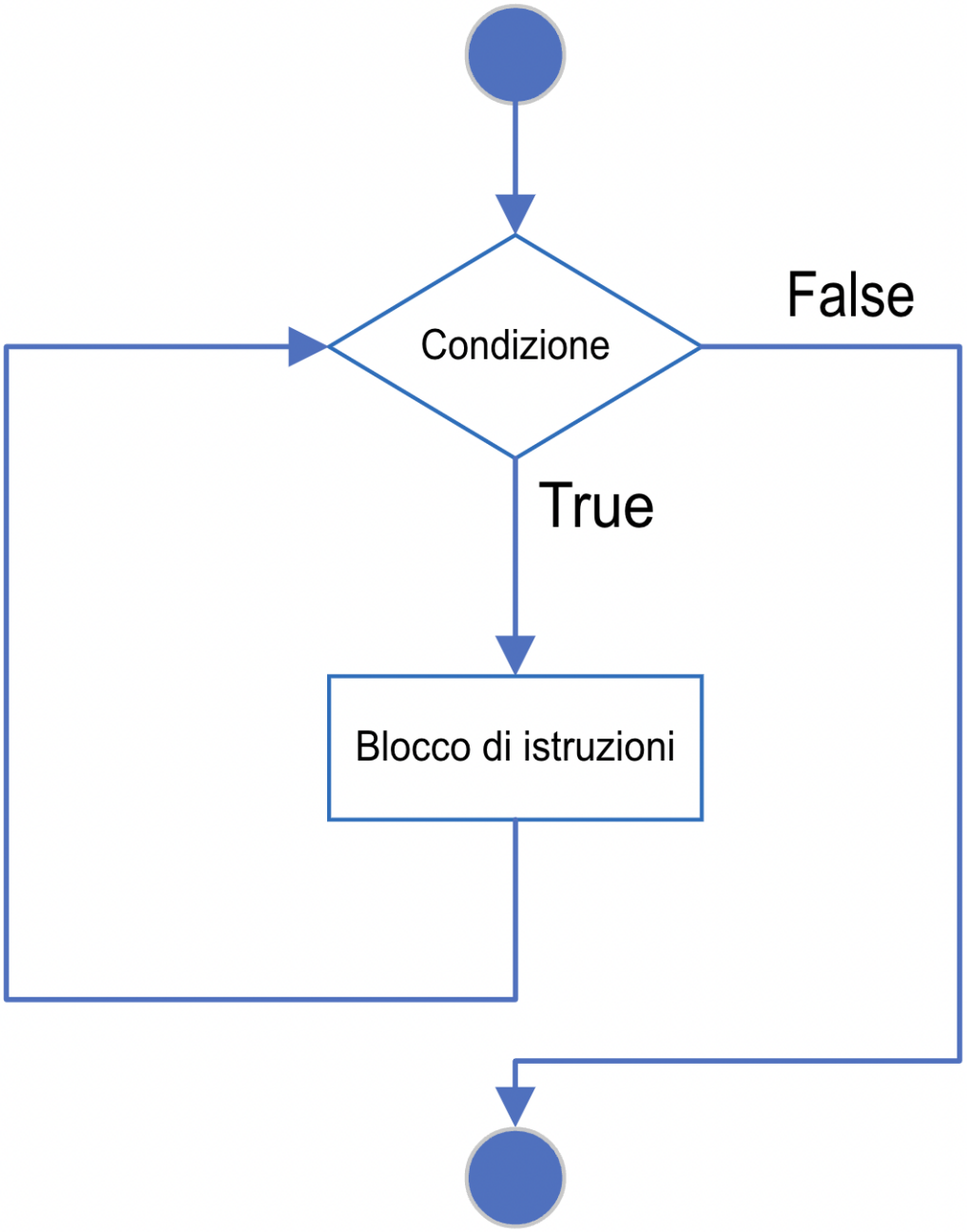
```
# TABELLINE - stampa la tabellina del 3

n_tabellina=3
i = 1
risultato= n_tabellina * i
print( n_tabellina, '*', i, '=', risultato)
i = 2
print( n_tabellina, '*', i, '=', risultato)
i = 3
print( n_tabellina, '*', i, '=', risultato)
```

```
i = 4
print( n_tabellina, '*', i, '=', risultato)
```

Se voglio stampare i primi 1000 valori questa soluzione è impraticabile…

Il costrutto `WHILE` risolve il problema



- La `condizione` viene valutata
- Se la `condizione` è vera, viene eseguito il corpo del **while**, e la `condizione` viene rivalutata
- Se la `condizione` è falsa, si esce dal ciclo

`WHILE` - sintassi semplificata

```
while (condizione): # finchè la condizione è vera...
    # corpo del while:
    # sequenza di istruzioni che vengono ripetute
    # finchè condizione è vera
# NEXT INSTRUCTION
```

- La `condizione` viene valutata
- Se la `condizione` è vera, viene eseguito il corpo del while, e la `condizione` viene rivalutata
- Se la `condizione` è falsa, si esce dal ciclo (il controllo passa a `# NEXT INSTRUCTION`)

NOTA:

Se la condizione è inizialmente `False` il corpo del `while` non verrà mai eseguito.

Se la condizione è inizialmente `True` ed il corpo del `while` non la altera, si entra in un loop infinito. **È fondamentale dunque che il corpo del `while` abbia la possibilità di alterare la condizione**.

```
while (True): # finchè la condizione è vera...
    print('fermatemiiii!')
```

Soluzione esercizio Tabellina

```
# i = 1
# finchè i è minore di 10 esegui le seguenti istruzioni:
#   calcola 3*i
#   stampa il risultato
#   incrementa i

i = 1
n_tabellina = 3
while ( i<=10 ):
    risultato =  n_tabellina *i
    print( n_tabellina, '*', i, '=', risultato)
    i = i+1
```

Schemi logici dei cicli `while`

Quando si utilizzano cicli per risolvere un problema è utile ragionare prima sulla **logica della ripetizione**, indipendentemente dal linguaggio di programmazione. Molti problemi che richiedono iterazione possono infatti essere ricondotti a pochi **schemi ricorrenti**. Riconoscere questi schemi aiuta a capire come organizzare il ragionamento prima ancora di scrivere il codice.

La logica del ciclo `while` è sempre la stessa: il corpo del ciclo viene eseguito finché la condizione rimane vera. Tuttavia è utile distinguere i diversi contesti logici in cui il ciclo viene utilizzato.

```
inizializzazione
while condizione:
    corpo del ciclo
    # durante il ciclo devono essere aggiornate
    # le variabili da cui dipende la condizione
```

Contesto	Idea
Numero di iterazioni noto in anticipo	ripetere un'operazione un numero noto di volte
Numero di iterazioni non noto in anticipo	ripetere un'operazione finché una condizione, aggiornata a ogni iterazione, rimane `True`

Nel primo schema il numero di iterazioni è noto o può essere determinato prima di iniziare il ciclo. Il ciclo viene quindi usato per eseguire un'operazione un certo numero di volte. Dal punto di vista logico è necessario inizializzare un contatore, verificare se il limite è stato raggiunto e aggiornare il contatore a ogni iterazione. Questo schema è tipico dei problemi in cui si devono elaborare un numero prefissato di valori.

Nel secondo schema il numero di ripetizioni non è noto in anticipo. Il ciclo continua finché una certa condizione rimane vera e termina quando la condizioen diventa falsa. Dal punto di vista logico l'attenzione non è sul numero di iterazioni ma sull'evoluzione dello stato del problema. A ogni iterazione alcune variabili vengono aggiornate e la condizione viene rivalutata.

While - Numero di iterazioni noto in anticipo

Utilizziamo un *contatore* che viene incrementato ad ogni iterazione ed utilizzato per valutare la condizione.

Esempio:

L'utente inserisce un numero che definisce il numero di iterazioni.

In seguito il programma chiede all'utente dei valori e li stampa

```
print('quanti numeri vuoi inserire?')
n = int(input())
i=0 # contatore delle iterazioni

while i < n:
    i = i + 1 # i viene incrementata
    # i += 1 # sintassi abbreviata
    valore = float(input())
    print('hai inserito il numero ', valore)
print('Istruzione fuori dal ciclo')
```

In questo esempio un contatore viene inizializzato a 0 e viene incrementato ad ogni esecuzione del corpo del `while`. La condizione `i<n` viene valutata all'inizio del ciclo (quando `i == 0`) e all'inizio di ogni iterazione.

Supponiamo che `n == 3`

Alla prima iterazione il valore di `i` passa da 0 a 1. Alla seconda iterazione il valore passa da 1 a 2. Alla terza iterazione il valore passa da 2 a 3. Al termine della terza iterazione la condizione `i < n` è `False` ed il ciclo si interrompe. Come previsto vengono eseguite `n == 3` iterazioni complete.

Errori comuni

```
n = int(input()) # dimenticare int()
i=0 # dimenticare di inizializzare i

while (i<n):
    # sbagliare la condizione: i<=n, i>n, i != n

    i += 1 # dimenticare di incrementare
    # ...
```

While - Numero di iterazioni non noto in anticipo

Possiamo utilizzare il costrutto while per ripetere un blocco di istruzioni finché una certa condizione rimane vera. Il numero di iterazioni non è noto in anticipo, ma dipende da una condizione che viene valutata a ogni iterazione.

```
while condizione:
    # Blocco di istruzioni ripetute finché la condizione è vera
    # aggiornamento delle variabili da cui dipende la condizione
```

Il ciclo continuerà a eseguire il blocco di codice finché la condizione rimane `True`. Non appena la condizione diventa `False`, l'esecuzione del ciclo si interrompe.

Esempio: inserisci numeri; 0 per terminare

Qui la condizione di stop non è legata al numero di iterazioni svolto, ma al valore inserito in input.

```
valore = int(input())
while valore != 0 :
    print('hai inserito', valore)
    valore = int(input())
print('Istruzione fuori dal ciclo')
```

- Il primo valore viene letto prima del ciclo, perché la variabile `valore` è necessaria per valutare la condizione
- Se il primo valore è 0, la condizione è subito `False` e il ciclo non viene eseguito
- Quando durante il ciclo viene inserito 0, la condizione diventa `False` e il ciclo termina
- L'istruzione `print('hai inserito', valore)` non viene eseguita per il valore 0

Esempio: inserisci numeri; 0 per terminare

È possibile modificare il ciclo in modo che anche il primo input sia all'interno del ciclo. Poiché `valore` compare nella condizione del ciclo, la variabile deve avere un valore iniziale prima di entrare nel ciclo. Possiamo quindi assegnare un valore iniziale che sia diverso da 0, in modo da forzare l'ingresso nel ciclo.

```
valore = 1
while(valore != 0):
    valore = int(input())
    print('hai inserito', valore)
print('Istruzione fuori dal ciclo')
```

In questo caso l'istruzione

`print('hai inserito', valore)`

viene eseguita anche nell'ultima iterazione, quando `valore == 0`.

Nota: impostare un valore arbitrario purché diverso da 0 (`valore = 1`) per forzare l'ingresso nel ciclo potrebbe generare confusione. Spesso in questi casi si preferisce assegnare un valore diverso da 0 ma anche intrinsecamente diverso da tutti gli altri possibili valori di input. La prima inizializzazione può essere allora `valore = None`, dove `None` rappresenta l'assenza di valore (oggetto nullo).

Da notare che i cicli con prima lettura dentro o fuori dal ciclo sono strutturalmente diversi:

```
Schema A - prima lettura fuori dal ciclo
leggi valore
while valore != 0:
    elabora valore
    leggi valore


Schema B - prima lettura dentro il ciclo
inizializza valore
while valore != 0:
    leggi valore
    elabora valore
```

Variabile di controllo del ciclo

Possiamo utilizzare una variabile di controllo del ciclo.

```
continua = True # variabile di controllo
while continua: # cioè: continua == True
    valore = int(input())
    print('hai inserito', valore)
    continua = (valore!= 0)
```

L'espressione `valore != 0` produce un valore booleano (`True` o `False`). Questo valore viene assegnato alla variabile `continua`, che determina se il ciclo prosegue oppure termina.

Nota: `continua = (valore != 0)` equivale a

```
if valore == 0:
    continua = False
else:
    continua = True
```

Obiettivo del ciclo

Le categorie precedenti riguardano il controllo del ciclo. Gli schemi che seguono riguardano invece lo **scopo dell'elaborazione** svolta nel ciclo. Lo stesso schema può comparire sia con un numero di iterazioni noto sia con un numero non noto.

Schema logico	Idea centrale
Conteggio di occorrenze	contare quante volte si verifica un evento
Rilevazione di evento (sentinella)	verificare se un evento accade almeno una volta
Accumulazione	costruire progressivamente un risultato aggregato
Ricerca di estremo	trovare il massimo o il minimo tra i valori osservati
Aggiornamento iterativo	modificare progressivamente uno stato fino a una soglia

- **Conteggio di occorrenze**

Durante l'esecuzione del ciclo spesso è necessario **contare quante volte si verifica un certo evento**. Questo porta allo schema del **conteggio di occorrenze**. A ogni iterazione si verifica una condizione e, se questa è soddisfatta, si incrementa un contatore. Al termine del ciclo il contatore rappresenta il numero di volte in cui l'evento si è verificato. Questo schema è molto comune quando si analizzano sequenze di valori o si classificano dati.

- **Rilevazione di evento (sentinella)**

Un caso leggermente diverso è la **rilevazione di evento**. Qui non interessa sapere quante volte si verifica una certa condizione, ma soltanto se accade **almeno una volta**. In questo schema si utilizza una variabile logica che inizialmente indica che l'evento non è stato osservato. Se durante il ciclo la condizione si verifica, lo stato viene aggiornato. Alla fine del ciclo si può stabilire se l'evento è mai accaduto.

- **Accumulazione**

Un altro schema molto frequente è l'**accumulazione**. In questo caso il ciclo serve a costruire progressivamente un risultato combinando i valori elaborati a ogni iterazione. Si parte da un valore iniziale e lo si aggiorna ripetutamente secondo una certa regola. Esempi tipici sono la somma di valori, il calcolo di un totale o la costruzione di un risultato aggregato. Molti problemi più complessi, come il calcolo di una media, derivano direttamente da questo schema.

- **Ricerca di estremo**

Un'ulteriore situazione ricorrente è la **ricerca di un estremo**, come il valore massimo o minimo. Il ciclo confronta ogni nuovo valore con il miglior risultato trovato finora e aggiorna il risultato quando viene individuato un valore migliore. Dal punto di vista logico questo schema richiede di mantenere memoria della migliore soluzione trovata durante l'iterazione.

- **Aggiornamento iterativo**

Infine, in alcuni problemi il ciclo serve ad **aggiornare progressivamente uno stato** finché non si raggiunge una certa soglia o condizione di stabilità. In questo schema ogni iterazione applica una trasformazione allo stato corrente. Il ciclo termina quando la situazione soddisfa il criterio di arresto. Questo tipo di iterazione è tipico dei processi di simulazione o di avvicinamento graduale a un risultato.

Esempi

Calcolo della media dei voti. Il numero di esami è noto.

- Controllo del ciclo: numero di iterazioni noto in anticipo
- Obiettivo dell'elaborazione: accumulazione

```
print('Quanti esami hai sostenuto?')
n = int(input())
i = 0 # contatore
s = 0 # accumulatore
while i<n:
    print('inserisci il voto')
    v = int (input())
    s = s + v
    i += 1  # incremento del contatore- equivale a i = i + 1

print ('sono uscito dal ciclo')

media = s / n
print('media = ', media)
```

Calcolo della media pesata dei voti. Il numero di esami è noto.

- Controllo del ciclo: numero di iterazioni noto in anticipo
- Obiettivo dell'elaborazione: accumulazione

```
print('Quanti esami hai sostenuto?')
n = int(input())
i = 0
somma_voti_per_crediti = 0 # accumulatore
somma_crediti =0  #accumulatore
```

```
while i<n:
    print('inserisci il voto')
    v = int (input())
    print('inserisci i crediti')
    c = int (input())
    somma_crediti = somma_crediti + c
    somma_voti_per_crediti = somma_voti_per_crediti + v*c

    i += 1

print ('sono uscito dal ciclo')

media_pesata = somma_voti_per_crediti / somma_crediti
print('media pesata = ', media_pesata)
```

Calcolo spesa totale. Criterio di stop: inserisci 0

- Controllo del ciclo: numero di iterazioni non noto in anticipo
- Obiettivo dell'elaborazione: Accumulazione

```
s = 0 # accumulatore

print('inserisci il primo importo')
v = int(input())

while v !=0:
    s = s + v
    print('inserisci importo; 0 per uscire')
    v = int(input())

print ('sono uscito dal ciclo')

print('spesa totale = ', s)
```

Calcolo spesa totale - solo importi interi. Criterio di stop: inserisci un carattere

- Controllo del ciclo: numero di iterazioni non noto in anticipo
- Obiettivo dell'elaborazione: Accumulazione

```
s = 0
print('inserisci il primo importo(solo cifre), oppure una lettera per terminare')
v = input() # non converto ancora la stringa in intero
```

```
while v.isdigit():
    s = s + int(v)
    print('inserisci importo (solo cifre), oppure una lettera per terminare')
    v = input()

print ('sono uscito dal ciclo')

print('spesa totale = ', s)
```

Note:

- `v.isdigit()` rende `True` se e solo se la stringa v rappresenta un intero, per es. `"45"`, altrimenti rende `False`
- vedremo in seguito come gestire anche le stringhe che rappresentano `float`

Soluzione alternativa:

```
s = 0
continua = True

while continua:
    print('inserisci importo (solo cifre), oppure una lettera per terminare')
    v = input()
    if v.isdigit():
        s = s + int(v)
    else:
        continua = False

print ('sono uscito dal ciclo')
print('spesa totale = ', s)
```

Inserimento interi. Criterio di stop: inserisci il carattere `x`.

Altri caratteri diversi da `x` verranno ignorati. Se viene inserito un carattere diverso da `x` il programma deve chiedere un altro input e riesaminarlo.

- Controllo del ciclo: numero di iterazioni non noto in anticipo
- Obiettivo dell'elaborazione: adattabile a molti scopi

```
# soluzione ERRATA

print("inserisci un numero intero oppure 'x' - altri caratteri non ammessi")
v = input()
if v !='x':
```

```
    if v.isdigit():
        v = int(v)
    else:
        print("hai inserito un carattere non ammesso")
        v = input()
        # a questo punto dovrei ricontrollare se v è digit oppure `x` in un ciclo potenzia
```

```
# soluzione CORRETTA
print("inserisci un numero intero oppure 'x' - altri caratteri non ammessi")
v = input()
while(v != 'x'):
    if v.isdigit():
        print(v)
    else:
        print("hai inserito un carattere non ammesso")
    v = input()
```

Scrivere un programma che legga 10 numeri e conti quanti sono positivi

- Controllo del ciclo: numero di iterazioni noto in anticipo
- Obiettivo dell'elaborazione: Conteggio di occorrenze

```
n = 10
i = 0
n_positivi = 0
while i<n:
    v = int (input())
    if v > 0:
        n_positivi += 1

    i += 1
print('hai inserito', n_positivi, 'numeri positivi')
```

Nota: In una espressione matematica i valori `True` e `False` vengono interpretati come `1` e `0`, quindi è possibile sostituire la struttura condizionale con un costrutto più cinciso.

```
#if v > 0:
#    n_positivi += 1

n_positivi += v>0 # True o False
```

Scrivere un programma che legga dei voti e conti quanti voti sono sufficienti. Criterio di stop: inserisci un carattere

- Controllo del ciclo: numero di iterazioni non noto in anticipo
- Obiettivo dell'elaborazione: Conteggio di occorrenze

```
voto_sufficienza = 6;
# 6 per scuole superiori; 18 per univ

n_sufficienti = 0

print('inserisci voti; carattere per terminare')
v = input()
while v.isdigit(): # è convertibile in intero?
    voto = int(v)
    n_sufficienti += voto >= voto_sufficienza
    # equivale a
    # if voto >= voto_sufficienza:
    #    n_sufficienti += 1

    v = input()

print('hai inserito ',n_sufficienti, 'voti sufficienti')
```

Scrivere un programma che legga dei voti e conti quanti voti sono sufficienti.

Criterio di stop: al massimo 10 voti, ma il programma termina prima se l'utente inserisce un carattere.

- Controllo del ciclo: numero di iterazioni massimo noto in anticipo, ma è presente un criterio di uscita anticipata
- Obiettivo dell'elaborazione: Conteggio di occorrenze

```
n = 10
i = 0

voto_sufficienza = 6
n_sufficienti = 0

print('Inserisci voti; un carattere per terminare')
v = input()

while v.isdigit() and i < n:
```

```
    voto = int(v)

    if voto >= voto_sufficienza:
        n_sufficienti += 1

    i += 1
    v = input()

print('Hai inserito', i, 'voti')
print('Hai inserito', n_sufficienti, 'voti sufficienti')
```

Esercizi su rilevazione di evento

Tutti gli esercizi su conteggio di occorrenze si possono trasformare in rilevazione di evento. Si usa una variabile `flag` al posto di un accumulatore.

Per esempio: su 10 voti, verificare se almeno un voto è maggiore di 27.

```
n = 10
i = 0
sentinella = 27
flag = False

print('Inserisci voti')

while i < n:
    voto = int(input())

    if voto > sentinella:
        flag = True

    i += 1

if flag:
    print ('almeno un voto è maggiore di ',sentinella )
else:
    print ('nessun voto è maggiore di ',sentinella )
```

Esercizi su accumulazione

- da svolgere con numero di iterazioni noto o non noto in anticipo, oppure con un numero di iterazioni massimo ma con un criterio di stop anticipato.

Esempi:

Calcolo della somma totale, della media aritmetica, pesata, armonica, della mediana, della varianza.

Esercizi su Ricerca di estremo

- da svolgere con numero di iterazioni noto o non noto in anticipo, oppure con un numero di iterazioni massimo ma con un criterio di stop anticipato.

Esempio: su 10 voti, trovare il voto massimo.

La variabile `voto_max` memorizza **il valore massimo osservato fino a quel momento**.

All'inizio `voto_max` viene inizializzato a `0`. A ogni iterazione il programma legge un nuovo voto e lo confronta con il massimo corrente. Se il nuovo voto è maggiore, `voto_max` viene aggiornato. In questo modo, durante l'esecuzione del ciclo, `voto_max` conserva sempre **il voto più alto incontrato finora**. Al termine delle `n` iterazioni, la variabile contiene **il massimo tra tutti i voti inseriti**.

```
i = 0
n = 10

voto_max = 0
while i<n:
    voto = int (input())
    if  voto > voto_max:
        voto_max = voto;

    i +=1
print('il voto massimo è ...', voto_max)
```

Stesso esercizio, ma esaminando 10 valori di temperatura. In questo caso non possiamo inizializzare a `0` la variabile `temp_max`. COsa succederebbe con temperature tutte negative?

Si legge la prima temperatura prima del ciclo e la si usa per inizializzare `temp_max`.

Successivamente, a ogni iterazione, la nuova temperatura viene confrontata con il massimo corrente. Se è maggiore, `temp_max` viene aggiornato. In questo modo la variabile conserva sempre la temperatura più alta osservata finora.

```
i = 0
n = 10

# prima lettura
temperatura = float(input())
```

```
temp_max = temperatura

i += 1

while i < n:
    temperatura = float(input())

    if temperatura > temp_max:
        temp_max = temperatura

    i += 1

print('la temperatura massima è ...', temp_max)
```

Altri Esercizi (non in ordine di difficoltà)

- fattoriale
- calcolo della media escliudet il voto più basso
- calcolo della media tranne il voto più basso e più alto
- calcolare i primio n elementi della sucessione di Fibonacci
- calcolo della spesa finale, applicando uno sconto a seconda di determinate condizoni
- sommare solo i valori che sddisfano una condizione
- indovinare un numero: l'utente deve indovinare un numero tra 1 e 100. Ad ogni tentativo il programma dice se il numero è stato indovinato, o se il numero inserito dall'utente è più grande o più piccolo del numero misterioso.
- Scrivere un programma che chiede all'utente un valore `n` e stampa i numeri PARI fino ad `n`. Se `n==10` il programma deve stampare `2, 4, 6, 8, 10`
- Scrivere un programma che chiede all'utente quante frasi desidera inserire. Il programma acquisisce le frasi e stampa la frase più lunga. Hint: usare `len(stringa)`

- Conversione base 2-> base 10. Scrivere un programma che chieda all'utente il numero di bit che intende utilizzare, e poi chieda il valore dei bit a partire dal *msb* (most significant bit, il bit più significativo). Il programma deve stampare il valore in base 10.

```
# SOLUZIONE:
print("Preparati ad inserire un numero intero espresso in base 2, a partire da MSB")
print("Quanti bit?")

n= int(input())
print("Ok, mi preparo a converitire un numero di ", n, " bit. Comincia l'inserimento")
i=0
v=0
while i<n:
    v= v*2
    b= int(input())
    v= v+b
    i+=1
print('In base 10: ---> ',v)
```

Gestire il ciclo con break e continue

Nel ciclo `while` possiamo usare due parole chiave speciali per controllare l'esecuzione.

- `break`: interrompe completamente il ciclo e passa all'istruzione successiva fuori dal `while`. Si noti che in caso di cicli annidati (*nested*), cioè inseriti uno dentro l'altro, il `break` interrompe solo il ciclo più interno.
- `continue`: salta l'iterazione corrente e passa all'iterazione successiva, senza eseguire il resto del codice all'interno del ciclo ma senza interrompere il ciclo

Uscire dal ciclo con `break`

Esempio: chiedere all'utente di inserire un numero finché non digita 0:

```
while True:  # Un ciclo teoricamente infinito
    print("Inserisci un numero (0 per uscire): ")
    numero = int(input())
    if numero == 0:
        print("Hai scelto di terminare il programma.")
        break  # Esce dal ciclo
    print("Hai inserito:", numero)
```

Saltare un'iterazione con `continue`

La parola chiave continue permette di saltare l'iterazione corrente e passare alla successiva.

Esempio: stampare i numeri da 1 a 10, ma saltare i multipli di 3

```
numero = 0

while numero < 10:
    numero += 1
    if numero % 3 == 0:
        # se il resto della divisione tra numero e 3 è ==0
        continue  # Salta il resto del codice e passa alla prossima iterazione
    print(numero)
```

`WHILE` - sintassi completa

https://docs.python.org/it/3.13/reference/compound_stmts.html#while

```
while condizione:
    # Blocco di codice eseguito finché la condizione è vera
else:
    # Blocco eseguito solo se il ciclo termina normalmente (senza break)
```

- Il blocco dentro `while` viene eseguito finché `condizione == True`.
- Se il ciclo termina normalmente (perché `condizione == False`), viene eseguito il blocco `else`.
- Se il ciclo viene interrotto con `break`, il blocco `else` non viene eseguito.

(IN PROGRESS)

Esercizi

- Scrivere un programma che contiene un numero segreto e chiede all'utente di indovinarlo. Sono ammessi 5 tentativi. La condizione di uscita deve quindi tener conto sia del valore inserito dall'utente sia del numero di tentativi.
- Scrivere un programma che chiede all'utente dei numeri e li somma. Il programma termina quando l'utente inserisce 0. Formulate una soluzione con il primo input prima del ciclo e un'alra soluzione cn tutti gli input all'interno del ciclo.

- Scrivere un programma che chiede all'utente dei numeri e li somma. Il programma termina quando la somma è maggiore di 100. Se dopo 10 iterazioni la somma non è ancora maggiore di 100, il programma termina lo stesso.

```
print('Inserisci i dati, termino quando la somma eccede 100 o dopo 10 inserimenti!')
s=0
i=0
while s<=100 and i<10:
    # DE MORGAN
    # not(s>100 or i>=10)
    v = int(input())
    s = s+v
    i += 1
print('somma:', s)
print('numero inserimenti:', i)

# VARIANTE:
# stampare un messaggio che spieghi se il ciclo è terminato
# perché somma >100 o perchè si sono superate le 10 iterazioni
```

- Scrivere un programma che chiede all'utente un insieme di numeri POSITIVI o NEGATIVI (ma non nulli) e ne calcola la somma. L'utente inserisce zero per uscire.
- Scrivere un programma che chiede all'utente un insieme di numeri qualunque e ne calcola la somma. L'utente inserisce il carattere `q` per uscire. Suggerimento: si supponga che l'utente non inserisca mai input errati. Se il valore inserito non è `q`, allora sicuramente è convertibile in numero.
- Scrivere un programma che chiede all'utente un insieme di numeri POSITIVI e ne calcola la media. L'utente inserisce zero per uscire. Variante: media robusta (eliminando il massimo e minimo)

- Dato un numero intero positivo, contare il numero di cifre. NB è richiesto di manipolare il numero, non la sua rappresentazione mediante stringa!

```
# Soluzione
print("Inserisci un numero intero, calcolerò il numero di cifre")
n= int(input())
i=0
while n>=1:
    i= i+1
    n = n/10
print('Numero di cifre ---> ',i)
```

- Dato un numero intero positivo, stampare il numero al contrario. NB è richiesto di manipolare il numero, non la sua rappresentazione mediante stringa!

```
# Soluzione
n_reverse=0
print('Inserisci un numero')
n= int(input())
while n>=1:
    resto= n%10
    n= int(n/10)
    n_reverse = n_reverse*10
    n_reverse = n_reverse + resto
print(n_reverse)
```

- Dati due numeri interi positivi, trovare il loro massimo comune divisore (MCD) usando l'Algoritmo di Euclide https://it.wikipedia.org/wiki/Algoritmo_di_Euclide

---

- Scrivere un programma che dica se p è un numero è primo. Varianti: se p è un numero primo, dire se è un numero primo di Sophie Germain[1]. Dire se è un numero primo di Mersenne[2]

- Scrivere un programma che contiene un numero segreto e chiede all'utente di indovinarlo. Variante: Il computer sceglie un numero segreto tra 1 e 100, e l'utente ha solo 5 tentativi per indovinarlo. Dopo ogni tentativo il programma deve dire "Troppo alto" o "Troppo basso" finché l'utente indovina o esaurisce i tentativi. Suggerimento: usa la funzione `random.randint()` per generare un numero casuale.

```
import random

# genera un numero (pseudo) casuale tra 1 e 100
numero_segreto = random.randint(1,100)

n= 5
i = 0
while i<n:
```

[1] https://it.wikipedia.org/wiki/Numero_primo_di_Sophie_Germain
[2] https://it.wikipedia.org/wiki/Numero_primo_di_Mersenne

```
    i = i+1
    print('indovina il numero')
    v = int(input())
    if v == numero_segreto:
        print('INDOVINATO!')
        break
    elif v<numero_segreto: # riferito a IF
        print('Hai inserito un numero troppo piccolo')
    else:
        print('Hai inserito un numero troppo grande')
else: # riferito a WHILE
    print('mi dispiace, hai esautito i tentativi')
    print('il numero era ', numero_segreto)


# La maggior parte dei linguaggi di programmazione hanno la struttura WHILE
# senza la clausola ELSE
# Formulate una soluzione alternativa SENZA la clausola ELSE del WHILE
# in modo che si possa implementare in altri linhguaggi di programmazione
```

ESERCIZI SUL WHILE

ESERCIZIO

Chiedi all'utente di inserire numeri finché digita 0. Salva i numeri in una lista e stampala alla fine.

```
numeri = []
print("Inserisci un numero (0 per terminare): ")
numero = int(input())
#numero = int(input("Inserisci un numero (0 per terminare): "))

while numero != 0:
    numeri.append(numero)
    numero = int(input("Inserisci un numero (0 per terminare): "))

print("Hai inserito:", numeri)
```

ESERCIZIO

Somma dei numeri pari

Obiettivo: Inserisci numeri finché l'utente scrive -1. Salva i numeri pari in una lista e calcola la loro somma.

```
pari = []
numero = int(input("Inserisci un numero (-1 per terminare): "))

while numero != -1:
    if numero % 2 == 0:
        pari.append(numero)
    numero = int(input("Inserisci un numero (-1 per terminare): "))

print("Numeri pari:", pari)

somma=0
i=0
while i<len(pari):
    somma += pari[i]
    i += 1


print("Somma dei pari:", somma))
```

ESERCIZIO

Lista al contrario

Chiedi n numeri all'utente, poi stampa la lista in ordine inverso usando while.

```
numeri = []
i = 0

n=5
while i < n:
    num = int(input(f"Inserisci il numero {i+1}: "))
    numeri.append(num)
    i += 1

# Stampa al contrario con while
i = len(numeri) - 1
print("Numeri al contrario:")
while i >= 0:
    print(numeri[i])
    i -= 1
```

```
# soluzione alternativa
i =  - 1
print("Numeri al contrario:")
while i >= -n:
    print(numeri[i])
    i -= 1
```

ESERCIZIO

Rimuovi tutti i negativi da una lista

Inserisci numeri finché digiti `q`, poi rimuovi tutti i numeri negativi dalla lista.

```
numeri = []
elemento = input("Inserisci un numero (q per terminare): ")

while elemento != 'q':

    numeri.append(int(elemento))
    elemento = input("Inserisci un numero (q per terminare): ")

# Rimozione negativi
# ATTENZIONE: incremento solo se non negativo!
i = 0
while i < len(numeri):
    if numeri[i] < 0:
        numeri.pop(i)
    else:
        i += 1

print("Lista senza numeri negativi:", numeri)
```

questo non funziona; perché?

```
i = 0
while i < len(numeri):
    if numeri[i] < 0:
        numeri.pop(i)

    i += 1
```

questo non funziona; perché?

```
i = 0
while i < len(numeri):
    idx= len(numeri)- i-1
    el = numeri[idx]
    if el < 0:
        #numeri.pop(idx)
        del numeri[idx]

    i += 1
print(numeri)
```

questo non funziona; perché?

```
i = -1
while i >= -len(numeri):
    el = numeri[i]
    print(el)
    if el < 0:
        #numeri.pop(idx)
        del numeri[i]


    i -= 1
print(numeri)
```

ESERCIZIO

Conta occorrenze di una parola

Data una lista di stringhe, chiedi una parola da cercare e stampa quante volte è presente in lista

```
parole = ['cane', 'gatto', 'gatto', 'topo', 'gatto']


cerca = input("Quale parola vuoi contare? ")
conteggio = 0
i = 0

while i < len(parole):
    if parole[i] == cerca:
        conteggio += 1
    i += 1
```

```
print(f"La parola '{cerca}' è stata inserita {conteggio} volte.")
```

ESERCIZIO

Elimina i duplicati

Inserisci numeri finché l'utente digita -1. Crea una nuova lista senza duplicati.

```
numeri = []
numero = int(input("Inserisci un numero (-1 per terminare): "))

while numero != -1:
    numeri.append(numero)
    numero = int(input("Inserisci un numero (-1 per terminare): "))

unici = []
i = 0
while i < len(numeri):
    if numeri[i] not in unici:
        unici.append(numeri[i])
    i += 1

print("Lista senza duplicati:", unici)
```

oppure, senza usare `not in`:

(provare con while-else o con variabile sentinella)

```
numeri = []
numero = int(input("Inserisci un numero (-1 per terminare): "))

while numero != -1:
    numeri.append(numero)
    numero = int(input("Inserisci un numero (-1 per terminare): "))

print('!')
unici = [numeri[0]]
i = 1
while i < len(numeri):
    j=0
    while j < len(unici):
        if numeri[i] == unici[j]:
            break
        j+=1
```

```
    else:
        unici.append(numeri[i])
    i += 1

print("Lista senza duplicati:", unici)
```

ESERCIZIO

Interrompi se inserisci 3 volte lo stesso numero. Il numero inserito deve essere compreso tra 0 e 99

(senza usare dizionari)

```
n = 100
numero = int(input("Inserisci un numero (-1 per terminare): "))
numeri=[0]*n # crea una lista di n elementi, tutti a 0
while numero !=-1:
    numeri[numero]+=1
    if numeri[numero] ==3:
        break
    numero = int(input("Inserisci un numero (-1 per terminare): "))
```

ESERCIZIO

Chiedi 10 numeri. Calcola la media dei numeri in posizione pari (indice 0, 2, 4…).

```
numeri = []
i = 0

while i < 10:
    num = int(input(f"Inserisci il numero {i+1}: "))
    numeri.append(num)
    i += 1

somma = 0
conta = 0
i = 0

while i < len(numeri):
    if i % 2 == 0:
        somma += numeri[i]
        conta += 1
    i += 1
```

```
media = somma / conta
print("Media dei numeri in posizione pari:", media)
```

ESERCIZIO

Palindroma o no?

Verifica se una lista forma una parola palindroma (uguale da sinistra e da destra).

```
frase = ['a', 'b', 'b', 'a']
# frase = ['a','l', 'b', 'a']

i = 0 #indice che parte da sinistra
j = len(frase) - 1 #indice che parte da destra
palindroma = True

while i < j:
    if frase[i] != frase[j]:
        palindroma = False
        break
    i += 1
    j -= 1


if palindroma:
    print("La parola è palindroma!")
else:
    print("La parola NON è palindroma.")
```

Struttura iterativa `FOR`

Struttura iterativa `FOR`

Questa parte è da svolgere DOPO aver visto il materiale sulle LISTE

Si riportano qui gli elementi essenziali della funzione `range()`

La funzione `range()` restituisce un oggetto iterabile che produce una sequenza di numeri interi. La sintassi è `range(start, stop, step)`

- `start`: valore iniziale della sequenza (incluso). Default: 0

- `stop`: valore finale (escluso)
- `step`: incremento tra un elemento e il successivo. Default: 1

L'oggetto `range` non è una lista, ma immagazzina i parametri essenziali per la creazione di una lista di intei (il primo elemento, lo step, l'ultimo elemento). L'oggetto `range` può essere convertito in una lista tramite il comando `list()`[3].

https://docs.python.org/2/library/functions.html#range

FOR

In Python, il ciclo `for` è uno **statement di controllo del flusso** che consente di iterare su una sequenza (iterable), come una *lista*, una *tupla*, un *dizionario*, una *stringa* o un oggetto che implementa il protocollo dell'iteratore.

Il ciclo `for` in Python è semanticamente diverso da quello in altri linguaggi come C o Java: non esegue iterazioni basate su un'espressione booleana (come fa invece il `while`), ma scorre direttamente gli elementi di un oggetto iterabile.

SINTASSI COMPLETA

```
for <variabile> in <iterabile>:
    <blocco_di_codice>
else:
    <blocco_opzionale_else>
```

La keyword `else` definisce un blocco di codice opzionale che viene eseguito una sola volta alla fine del ciclo, *solo se il ciclo non è stato interrotto da un break*.

Gestire il ciclo con break e continue

Nel ciclo `for`, esattamente come in `while`, possiamo usare due parole chiave speciali per controllare l'esecuzione.

- `break`: interrompe completamente il ciclo e passa all'istruzione successiva fuori dal `for`. Si noti che in caso di cicli annidati (*nested*), cioè inseriti uno dentro l'altro, il `break` interrompe solo il ciclo più interno.
- `continue`: salta l'iterazione corrente e passa all'iterazione successiva, senza eseguire il resto del codice all'interno del ciclo ma senza interrompere il ciclo

[3] tecnicamente `list` è una classe built-in

ESERCIZI SUL FOR

Questa parte è da svolgere DOPO aver visto il materiale sulle LISTE

Alcuni esercizi sono gli stessi proposti per il while (o molto simili). Notate come la soluzione di alcuni esercizi è più semplice se uso il `FOR` invece del `WHILE`

ESERCIZIO

Stampa la lista in ordine inverso usando FOR

```
numeri = [1, 2,3,4,5]
n=len(numeri)
for i in range(n-1,-1,-1):
    print(numeri[i])
```

ESERCIZIO

Conta occorrenze di una parola

Data una lista di stringhe, chiedi una parola da cercare e stampa quante volte è presente in lista

```
parole = ['cane', 'gatto', 'gatto', 'topo', 'gatto']

cerca = input("Quale parola vuoi contare? ")
conteggio = 0

for el in parole:
    if el == cerca:
        conteggio += 1

print(f"La parola '{cerca}' è stata inserita {conteggio} volte.")
```

ESERCIZIO

Crea una nuova lista senza duplicati.

```
numeri = [1,2,3,2,3,4,3,4,5,6,7]

unici = []

for el in numeri:
```

```
    if el not in unici:
        unici.append(el)

print("Lista senza duplicati:", unici)
```

oppure, senza usare `not in`:

```
numeri = [1,2,3,2,3,4,3,4,5,6,7]

unici = []

for el_1 in numeri:
    for el_2 in unici:
        if el_1 == el_2:
            break
    else:
        unici.append(el_1)

print("Lista senza duplicati:", unici)
```

ESERCIZIO

Calcola la media dei numeri in posizione pari (indice 0, 2, 4…).

```
numeri = [1, 4, 3 , 8, 5, 3, 7, 1, 9] # mean == 5
somma=0
conta=0
for i in range(0,len(numeri),2):
    somma += numeri[i]
    conta+=1
media = somma/conta

print("Media dei numeri in posizione pari:", media)
```

ESERCIZIO

Palindroma o no?

Verifica se una lista forma una parola palindroma (uguale da sinistra e da destra).

```
frase = ['a', 'b', 'b', 'a']
# frase = ['a','l', 'b', 'a']

for i in range(0, int(len(frase)/2)):
    j = len(frase)-i-1
    if frase[i] != frase[j]:
        print("La parola NON è palindroma.")
        break
else:
    print("La parola è palindroma!")
```

ESERCIZIO

Data una lista, conta il numero di elementi pari e elementi dispari

```
lista = [ 1,2,3,4,5,6,7,8,9]

n_pari = 0
n_dispari = 0

for el in lista:
    if el%2==0:
        n_pari += 1
    else:
        n_dispari += 1

print('pari: ', n_pari, 'dispari: ', n_dispari )
```

ESERCIZIO

Data una lista, stampa gli elementi maggiori di 10

```
lista = [ 1,3,5,10, 21, 44]

for el in lista:
    if el>10:
        print (el)
```

ESERCIZIO

Data una lista, crea una nuova lista raddoppiando i valori negativi e lasciando inalterati i valori positivi

```
lista = [ 1,-3,5,10, -4, 4]
lista2=[]
for el in lista:
    if el>0:
        lista2.append(el)
    else:
        lista2.append(2*el)
print(lista2)
```

ESERCIZIO

Data una lista, raddoppia i valori negativi e lascia inalterati i valori positivi

```
lista = [ 1,-3,5,10, -4, 4]

for i in range(len( lista)):
    if lista[i]<0:
        lista[i]*=2
print(lista)
```

ESERCIZIO

Trova la posizione del primo numero negativo

```
lista = [ 1,-3,5,10, -4, 4]
# lista = [1,2,3,4,5]

trovato = False
for i in range(len( lista)):
    if lista[i]<0:
        i_neg = i
        trovato = True
        break
if trovato:
    print ('posizione del primo numero negativo: ',i_neg)
else:
    print('solo valori positivi')
```

ESERCIZIO

Conta le vocali in una lista di lettere

```
lettere = ['l', 'e', 'o', 'p', 'a', 'r', 'd', 'o']
vocali = ['a', 'e', 'i', 'o', 'u']
conteggio = 0

for l in lettere:
    if l in vocali:
        conteggio += 1

print("Numero di vocali:", conteggio)
```

ESERCIZIO

Confronta due liste e trova gli elementi comuni

Il programma deve produrre una lista di elementi comuni NON duplicati

```
lista1 = [7, 11, 13, 19]
lista2 = [2,3,5,7, 19, 23,29]
comuni = []

for el in lista1:
    if el in lista2 and el not in comuni:
        comuni.append(el  )

print("Elementi comuni:", comuni)
```

Soluzione che non usa l'operatore `in lista`. Il codice esegue cicli non necessari (analizzate il codice passo-passo). COME SI PUO' MIGLIORARE?

```
lista1 = [7, 11, 13, 19, 7]
lista2 = [2, 3, 5, 7, 19, 23, 29]
comuni = []

for el1 in lista1:
    for el2 in lista2:
        if el1 != el2:
            continue
        else: # el1==el2
            for el3 in comuni:
                if el1 == el3:  # esiste già
                    break
            else: # raggiunge il termine naturale del for senza break
```

```
                comuni.append(el1)

print("Elementi comuni:", comuni)
```

Codice migliorato:

```
lista1 = [7, 11, 13, 19, 7]
lista2 = [2, 3, 5, 7, 19, 23, 29]

comuni = []
for el1 in lista1:
    for el2 in lista2:
        if el1 == el2:
            for el3 in comuni:
                if el1 == el3:
                    break  # già presente
            else:
                comuni.append(el1)  # non è stato trovato, quindi lo aggiungiamo
            break  # usciamo da el2 - corrispondenza già trovata

print("Elementi comuni:", comuni)
```

ITERARE SUI DIZIONARI - ESERCIZI

in progress

Questa parte è da svolgere DOPO aver visto il materiale sui dizionari.

→ **Esercizio**.

Considerate due liste `l1` ed `l2` della stessa lunghezza. La lista `l1`contiene solo elementi non mutabili, unici. Scrivete un programma che, iterando su entrambe le liste, crea un dizionario usando come chiavi gli elementi della prima lista e come valore gli omologhi elementi della seconda lista

→ **Esercizio**. Dato un dizionario, creare una lista con i suoi valori. Utilizzate sia il metodo `values()` sia l'accesso ai valori tramite chavi e iterazioni.

→ **Esercizio**.

Considerate due liste `l1` ed `l2` della stessa lunghezza, contenenti rispettivamente città e temperature. Le città possono essere ripetute. Questo significa che per quella città la temperatura è stata rilevata più volte.

```
l1= ['ca', 'ge', 'ca']
l2= [30, 27, 28]
```

create un dizionario con chiave il nome della città e come valori la *lista di temperature*. Nel caso la città compaia solo una volta, il valore deve essere comunque una lista ocn un solo elemento

→ **Esercizio**.

scandite un dizionario. Il ciclo si deve interrompere quando `chiave == 0`

Varianti: - condizioni analoghe per il valore. - se una condizione diventa `True`, la chiave successiva deve essere saltata